UNIVERSITÀ DEGLI STUDI DI BARI ALDO MORO

ITPS
gruppo 31

ENGINEERING SUPPORT

797163. Faliero Samuele Belardinelli, f.belardinelli@studenti.uniba.it
801933. Ennio Lo Buono, e.lobuono@studenti.uniba.it

DATA DI RILASCIO: 16/12/2025
PROGRAMMAZIONE PER IL WEB ANNO 2025/2026

SOMMARIO

1. DESCRIZIONE INFORMALE DEL CONTESTO E DEGLI OBIETTIVI . 3
2. INDIVIDUAZIONE DEGLI STAKEHOLDER E DEI GOAL 3
2.1 Individuazione goal amministratore 4
2.2 Diagramma di contesto 5
2.3 Scambio di valore 5
3. STATO DELL'ARTE, ANALISI DELLA CONCORRENZA E ANALISI "MAKE OR BUY" 6
3.1 Stato dell'arte 6
3.2 Analisi della concorrenza 6
3.3 Analisi "Make or Buy" 8
4. MODELLO INFORMATIVO 8
4.1 Diagramma EER 9
4.2 Modello relazionale 9
4.3 Stima dimensionale del DB 11
4.3.1 Stima dimensionale per entità 11
4.3.2 Tabella popolamento iniziale Errore. Il segnalibro non è definito.
4.3.3 Crescita annuale 12
4.4 Modello di visibilità 12
5. MODELLO DI NAVIGAZIONE E PRESENTAZIONE 15
5.1 Modello di navigazione 15

5.2 Modello di presentazione 15
6. MODELLO ARCHITETTURALE 17
6.1 Hardware 17
6.2 Software 17
6.2.1 Application server 17
6.2.2 Librerie principali 17
6.2.3 Database Management System 18
6.2.4 Browser supportati 18
6.2.5 Sistemi operativi supportati 18
7. TEST 19
8. CONCLUSIONI 20
9. MATRICE RACI 21

1. DESCRIZIONE INFORMALE DEL CONTESTO E DEGLI OBIETTIVI

La nostra web application nasce come **strumento di supporto alla progettazione software**, pensato per aiutare i team di sviluppo a documentare il processo in modo strutturato, coerente e tracciabile, senza doversi preoccupare di "tenere insieme" manualmente le informazioni nelle diverse sezioni di uno stesso progetto.
In un contesto di **progetti software di media e grande complessità**, in cui requisiti, casi d'uso, modelli dati e diagrammi aumentano e si differenziano rapidamente, la documentazione rischia spesso di diventare frammentata, ridondante o non aggiornata. La nostra applicazione si posiziona esattamente qui: come **fulcro,** in cui gli item di progetto vengono raccolti, validati e collegati in modo automatico, garantendo coerenza interna e facilità di consultazione per tutti i partecipanti attivi e passivi.

- **Contesto applicativo,** l'applicazione è pensata per ambienti in cui sono richiesti:
  - **documenti di progetto dettagliati e tracciabili** (requisiti, casi d'uso, specifiche tecniche, modelli dati);
  - **allineamento continuo** tra analisi, progettazione e implementazione;
  - **collaborazione multiruolo** (sviluppatori, PM, QA, architetti) su uno stesso set di informazioni;
  - **conformità a linee guida e standard interni** o normativi sulla documentazione.

Invece di mantenere testi, schemi e diagrammi in strumenti separati, l'applicazione centralizza i contenuti e ne gestisce in automatico i collegamenti logici, riducendo il rischio di errori, incoerenze e duplicazioni.

- **Obiettivi dell'applicazione,** L'obiettivo principale è **rendere la fase di analisi e progettazione dati più veloce, affidabile e "a prova di errore umano"**. In particolare, la web application si propone di:
  - **Guidare** il progettista nella compilazione dei documenti, assicurando che ogni sezione contenga le informazioni minime necessarie.
  - **Collegare automaticamente** determinati artefatti di progetto (requisiti, casi d'uso, entità, attributi, relazioni, diagrammi) mantenendo la coerenza lungo tutto il ciclo di vita.
  - **Generare in modo automatico**:
    - schemi e documentazione tecnica a partire dagli item inseriti;
    - modelli logici e fisici di database, pronti per l'implementazione;
    - diagrammi dimostrativi (ad es. schema logico, diagrammi di sequenza, riferimenti a classi e class diagram).
  - **Ridurre drasticamente** inconsistenze, omissioni e dimenticanze durante la progettazione, grazie a controlli di validazione e notifiche guidate.
  - **Abilitare l'esportazione** dei contenuti in formati standard (es. PDF, DOCX, LaTeX), facilitando condivisione, revisione e archiviazione.

In sintesi, la nostra applicazione si presenta come una **piattaforma di strumentazione della documentazione di progetto**: trasforma le descrizioni fornite dagli utenti in documentazione strutturata, modelli di database e artefatti tecnici coerenti, diventando un vero **acceleratore di analisi e progettazione** per i team di ingegneria del software.

2. INDIVIDUAZIONE DEGLI STAKEHOLDER E DEI GOAL

2.1 Individuazione goal amministratore

L'applicazione si colloca in un contesto di **progettazione software collaborativa**, in cui più figure professionali interagiscono sulla stessa base informativa.
Gli stakeholder principali sono:

- **Ingegneri del software / sviluppatori** – compilano gli item di progetto (requisiti, casi d'uso, modelli dati) e utilizzano la piattaforma per generare documentazione e schemi tecnici.
- **Product e Project Manager (PM)** – monitorano l'avanzamento, verificano coerenza e completezza della documentazione e coordinano il lavoro del team.
- **Amministratore della piattaforma** – configura il sistema, gestisce utenti, ruoli, permessi e supervisiona la qualità complessiva dei dati.

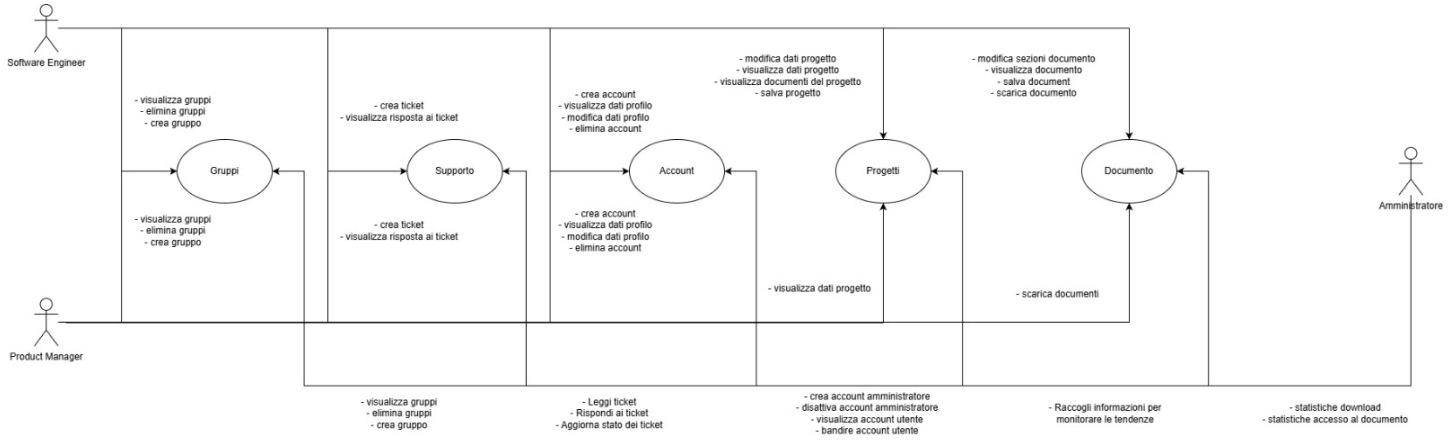
L'obiettivo comune è disporre di una **fonte unica, coerente e tracciabile** per tutti gli artefatti di analisi e progettazione, riducendo attività manuali e margini di errore.

STAKEHOLDER	GOAL	SUBGOAL	C	R	U	D
AMMINSTRATORE	gestione dati informativi	visualizza statistiche accessi		x		
		visualizza statistiche download		x		
		visualizza statistiche di accesso ad un documento		x		
		visualizza statistiche relative ad un progetto		x		
	gestione account	crea account amministratore	x			
		visualizza account utente		x		
		disattiva account amministratore			x	
		bandisci account utente				x
	gestione supporto	rispondi ai ticket	x			
		leggi ticket		x		
		aggiorna stato ticket			x	
	gestione gruppi	crea gruppo	x			
		visualizza gruppo		x		
		elimina gruppo				x

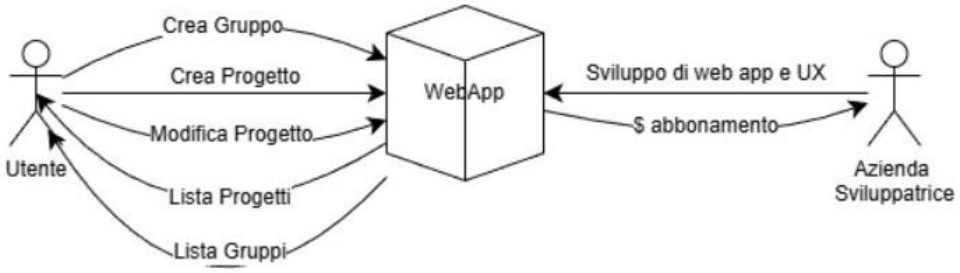
STAKEHOLDER	GOAL	SUBGOAL	C	R	U	D
PRODUCT MANAGER	gestione progetto	visualizza dati progetto		x		
	gestione account	crea account	x			
		visualizza dati profilo		x		
		modifica dati profilo			x	
		elimina account				x
	gestione supporto	crea ticket	x			
		visualizza risposta al ticket		x		
	gestione gruppi	crea gruppo	x			
		visualizza gruppo		x		
		elimina gruppo				x
	gestione documento	scarica documento		x		

STAKEHOLDER	GOAL	SUBGOAL	C	R	U	D
SOFTWARE ENGINEER	gestione progetto	crea progetto	x			
		visualizza dati progetto		x		
		salva progetto			x	
		elimina progetto				x
	gestione account	crea account	x			
		visualizza dati profilo		x		
		modifica dati profilo			x	
		elimina account				x
	gestione supporto	crea ticket	x			
		visualizza risposta al ticket		x		
	gestione gruppi	crea gruppo	x			
		visualizza gruppo		x		
		elimina gruppo				x
	gestione documento	crea documento	x			
		visualizza documento		x		
		scarica documento		x		
		modifica documento			x	
		salva documento			x	
		elimina documento				x

2.2 Diagramma di contesto



2.3 Scambio di valore



3 STATO DELL'ARTE, ANALISI DELLA CONCORRENZA E ANALISI "MAKE OR BUY"

3.1 Stato dell'arte

Negli ultimi anni gli strumenti a supporto dell'ingegneria del software hanno fatto un salto di qualità importante: da semplici repository di requisiti o documenti statici si sono trasformati in **piattaforme integrate di Application Lifecycle Management (ALM)**, in grado di coprire analisi, progettazione, sviluppo e manutenzione.
Le principali tendenze di riferimento sono:

- **Centralizzazione delle informazioni di progetto**
  Requisiti, casi d'uso, backlog, modello dati e documentazione non sono più dispersi tra file Word, Excel e diagrammi scollegati, ma vengono gestiti in **un unico ambiente** con relazioni esplicite tra gli artefatti.
- **Integrazione con strumenti di sviluppo e DevOps**
  Le piattaforme moderne si integrano con sistemi di versioning (che permette di tenere traccia nel tempo), CI/CD (che automatizza il build/test/deploy) e issue tracking (che organizza lavoro e bug), creando un flusso continuo tra analisi, sviluppo e rilascio. La documentazione non è più un "mondo a parte", ma dialoga con il codice.
- **Documentazione "vivente" e approccio doc-as-code**
  Si diffondono approcci in cui la documentazione è versionata insieme al codice, generata da template e automatismi. L'obiettivo è ridurre disallineamenti, ridondanze, attività manuali a basso valore aggiunto ed errori superflui e accidentali.
- **Supporto alla collaborazione multiruolo**
  Gli strumenti sono pensati per diversi team di lavoro (come: analisti, sviluppatori, QA, stakeholder di business), con viste e permessi differenti, ma basati sugli stessi dati. Questo è importante per progetti con molti attori e responsabilità distribuite.
- **Automazione tramite template, regole e AI**
  Molti strumenti offrono scorciatoie per scrivere documenti di progetto: modelli pronti da compilare, checklist usati come promemoria e controlli automatici per la segnalazione di errori o incongruenze. Queste funzioni sono utili perché supportano interamente il progetto. Tuttavia, queste capacità sono spesso generiche e non sempre adatte ad un percorso strutturato. L'obbiettivo, quindi, è di seguire un flusso preciso requisiti → casi d'uso → modello concettuale → schema di database, con un supporto manuale per adattare i suggerimenti e mantenere tutto coerente con il metodo e modello richiesto.

In questo scenario, il progetto sviluppato nell'ambito del corso di **Programmazione per il Web** si posiziona come:
prototipo di web application specializzata per la documentazione di progetti software, focalizzato su una modellazione strutturata di requisiti, casi d'uso e modello dati, con collegamenti automatici tra artefatti e generazione di documentazione tecnica a partire dagli item inseriti.
L'obiettivo non è competere con i grandi player enterprise, ma **coprire in modo mirato il segmento dei progetti didattici / PMI**, dove servono:

- struttura e tracciabilità "da grande azienda",
- ma con **bassa complessità**, costi contenuti e curva di apprendimento compatibile con team piccoli o in formazione.

3.2 Analisi della concorrenza

La concorrenza non è rappresentata da un singolo prodotto, ma da categorie di soluzioni, che unite tra loro, portano un probabile competitor.

1. Piattaforme ALM / DevOps (Jira + Confluence, Azure DevOps, GitLab + Wiki, ...)
   - Punti di forza
     - Gestione strutturata di issue, user story e backlog.
     - Integrazione nativa con repository di codice e pipeline DevOps.

        - Ampia diffusione, community e plugin.
    - Limiti rispetto al nostro caso d'uso
        - Meta-modello centrato su task/issue, non su entità, attributi, relazioni e casi d'uso formalizzati.
        - Generazione di schema logico/fisico di database e documentazione di progetto molto limitata o inesistente.
2. Strumenti di requirements management enterprise (IBM DOORS Next, Polarion, Jama, ...)
    - Punti di forza
        - Gestione avanzata di requisiti, versioning e tracciabilità fine.
        - Workflow configurabili, supporto a normative e audit.
    - Limiti
        - Costi e complessità di adozione elevati, poco compatibili con contesti didattici o PMI.
        - Focalizzazione forte su requisiti testuali e compliance, meno su modellazione concettuale e database.
        - Bassa aderenza a uno scenario di prototipo accademico di Programmazione del Web.
3. Tool di produttività e knowledge management (Notion, Confluence, ClickUp, Google Workspace, ...)
    - Punti di forza
        - UI moderna, alta flessibilità, facilità di adozione.
        - Buon supporto alla collaborazione e alla condivisione.
    - Limiti
        - Struttura dati generica: il meta-modello di progetto software va "inventato" dal team.
        - Tracciabilità basata su link manuali e proprietà custom.
        - Nessun motore nativo per generare diagrammi EER, schema logico/fisico e documentazione tecnica a partire dagli item inseriti.
4. Strumenti di modellazione e diagrammazione (Enterprise Architect, draw.io, Lucidchart, ...)
    - Punti di forza
        - Ampio supporto a UML e BPMN.
        - Adatti a modellare sistemi anche complessi.
    - Limiti
        - Diagrammi gestiti come oggetti separati dalla documentazione testuale.
        - Coerenza tra testo e modelli demandata al singolo progettista.
        - Mancanza di un flusso guidato requisiti → casi d'uso → modello dati → documentazione integrata.
5. Soluzioni custom interne (template Word/Excel, wiki, script ad hoc)
    - Punti di forza
        - Nessun costo di licenza, massima libertà di struttura.
        - Possibilità di adeguare i template alle esigenze del corso o dell'azienda.
    - Limiti
        - Elevato rischio di inconsistenze, duplicazioni e perdita di dati per mancato salvataggio.
        - Forte dipendenza dalle pratiche informali del team.

Posizionamento del prototipo

La nostra proposta si colloca in uno spazio intermedio:

- più strutturato e guidato rispetto a Notion/Confluence e ai semplici template Word/Excel;
- molto più semplice, leggero e focalizzato rispetto alle piattaforme enterprise di ALM/requirements management.

Il vantaggio competitivo atteso è la combinazione di:

- meta-modello chiaro e didatticamente utile (requisiti, casi d'uso, entità, attributi, relazioni);
- collegamenti automatici tra artefatti;
- generazione semi-automatica di documentazione e schemi di database, pensata sia per la didattica sia per piccoli team di progetto.

3.3 Analisi “Make or Buy”

Per definire la strategia di implementazione è stata svolta una valutazione “Make or Buy”.

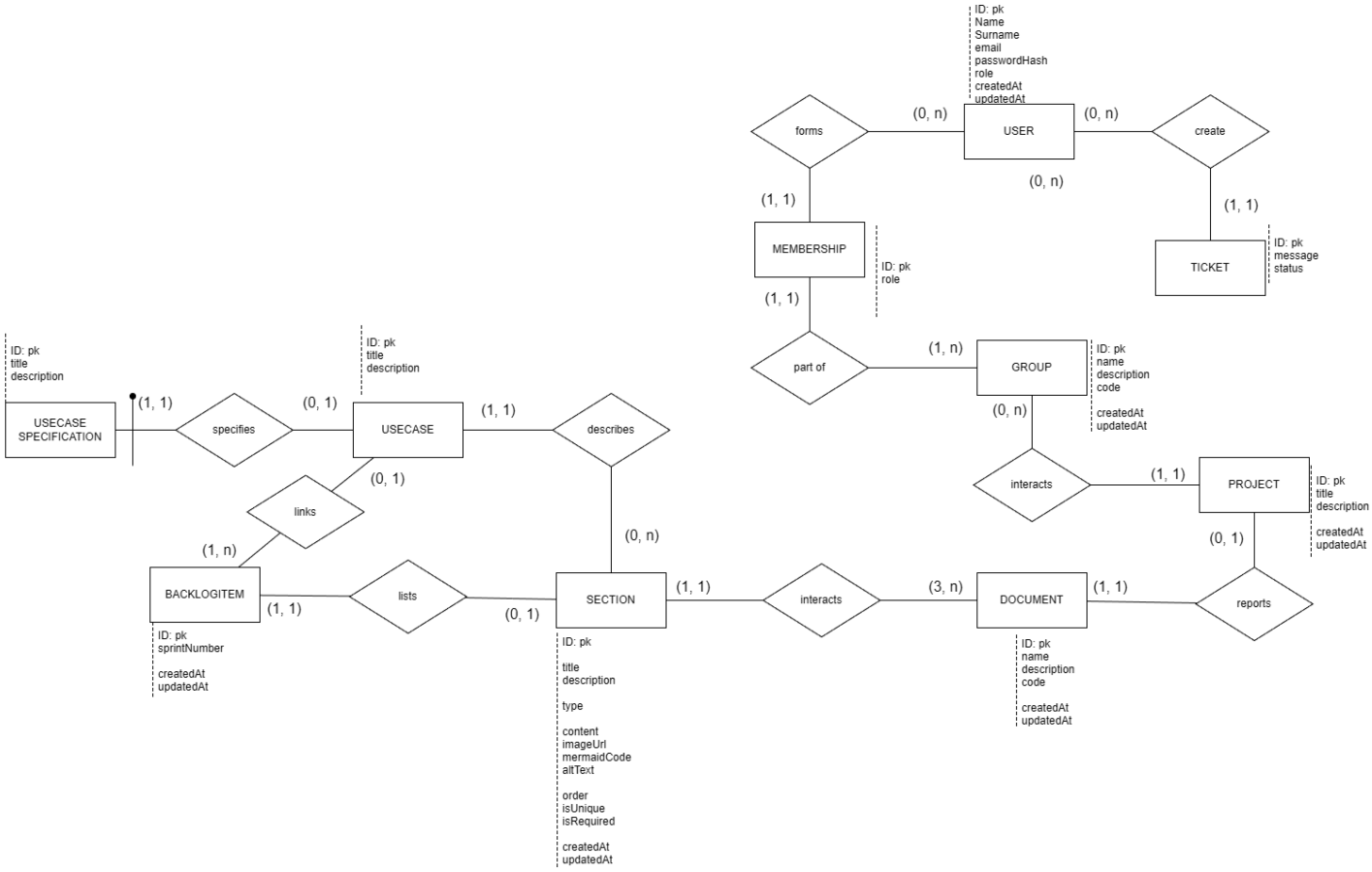
L’opzione Buy consiste nell’adottare e configurare una piattaforma già esistente (ad esempio strumenti di gestione requisiti e documentazione come Jira/Confluence, Azure DevOps, Notion o soluzioni analoghe). Questo approccio permetterebbe di partire più rapidamente sfruttando funzionalità già pronte come autenticazione, gestione utenti, permessi, spazi di lavoro, wiki e integrazioni di base con altri tool. Tuttavia, queste piattaforme sono progettate principalmente per la gestione di issue, backlog o pagine di documentazione generica e non per un meta-modello strutturato che colleghi in modo esplicito requisiti, casi d’uso, entità, attributi, relazioni e derivati (diagrammi, schema logico/fisico di database, documentazione tecnica).
Ricondurre il modello del progetto a questi strumenti richiederebbe personalizzazioni profonde (template, campi custom, automazioni, plugin), che potrebbero portare a riscrizioni di interi pezzi di codice e quindi di un investimento di tempo non indifferente. A questo si aggiungono vincoli legati a licenze, roadmap del fornitore e limiti delle API, oltre al fatto che l’utilizzo di una soluzione pre-esistente ridurrebbe il valore formativo legato alla progettazione e allo sviluppo end-to-end di una web application nel contesto del corso.

L’opzione Make prevede invece lo sviluppo di una web application dedicata, concepita fin dall’inizio come prototipo didattico per il corso di Programmazione per il Web e allineata al meta-modello richiesto. Questo approccio consente di modellare in modo esplicito gli artefatti di progetto (requisiti, casi d’uso, entità, attributi, relazioni) e di progettare il flusso che li collega, integrando nel codice le regole di coerenza e i vincoli di qualità che si vogliono imporre.
Inoltre, permette di scegliere lo stack tecnologico più adatto agli obiettivi del corso, di definire con precisione la struttura del database e di introdurre gradualmente funzionalità distintive come la generazione automatica della documentazione tecnica, l’esportazione degli schemi di database o l’eventuale integrazione futura di moduli di intelligenza artificiale.
Il prototipo può essere riutilizzato come caso di studio in altri anni accademici o esteso a contesti aziendali reali. La creazione “da zero” del progetto, non comporta un costo di licenza e di utilizzo, ma solamente un investimento iniziale maggiore in termini di analisi, progettazione e sviluppo, oltre alla necessità di presidiare internamente le scelte architetturali e le attività di manutenzione.

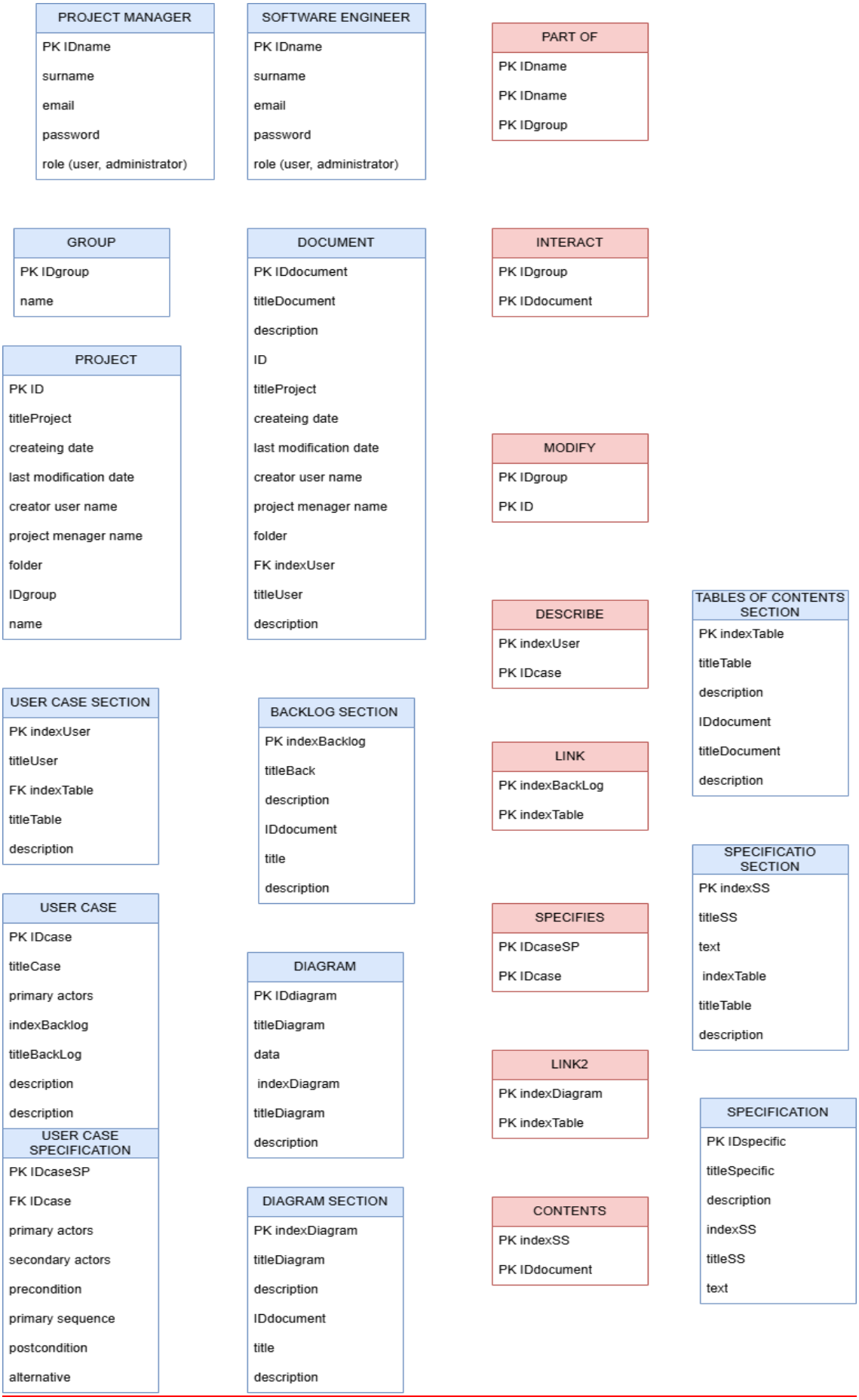
Alla luce di quanto emerso, ossia che le soluzioni “Buy” non offrono il meta-modello richiesto né la generazione automatica degli artefatti chiave del progetto, e che le personalizzazioni necessarie su piattaforme esistenti sarebbero onerose e solo parzialmente efficaci, si è deciso di seguire l’approccio Make. Sviluppare una soluzione dedicata permette di massimizzare il valore didattico del progetto, di mantenere il pieno controllo sulle scelte architetturali e di ottenere un prototipo coerente con gli obiettivi del corso e con il posizionamento individuato rispetto allo stato dell’arte e alle alternative presenti sul mercato. Rendere il progetto gratuito per il primo utilizzo e a pagamento per gli ulteriori utilizzi, è una decisione interna, col fine di posizionarsi strategicamente con un prodotto semplice ma efficace che, risultato dell’analisi svolta finora, non esiste, o meglio, esiste sfruttando l’operatività di molteplici strumenti.

4 MODELLO INFORMATIVO

4.1 Diagramma ER



4.2 Modello relazionale



4.3 Stima dimensionale del DB

4.3.1 Stima dimensionale per entità

Parametro	Valore stimato
Numero totale utenti	2.000
- Owners (10%)	200
- Product Managers (20%)	400
- Software Engineers (80%)	1400
- Utenti Base (95%)	1900
- Utenti Upgrade (5%)	100
Numero totale di Gruppi	500
Media utenti per gruppo	~4
Media progetti per gruppo	~4
Media sezioni per progetto	8
- Media di casi d'uso per progetto	35
- Media di specifiche per progetto	35
- Media di item del backlog per progetto	35
Numero di tickets	250
Periodo di conservazione	4 anni

4.3.2 Stima dello spazio occupato

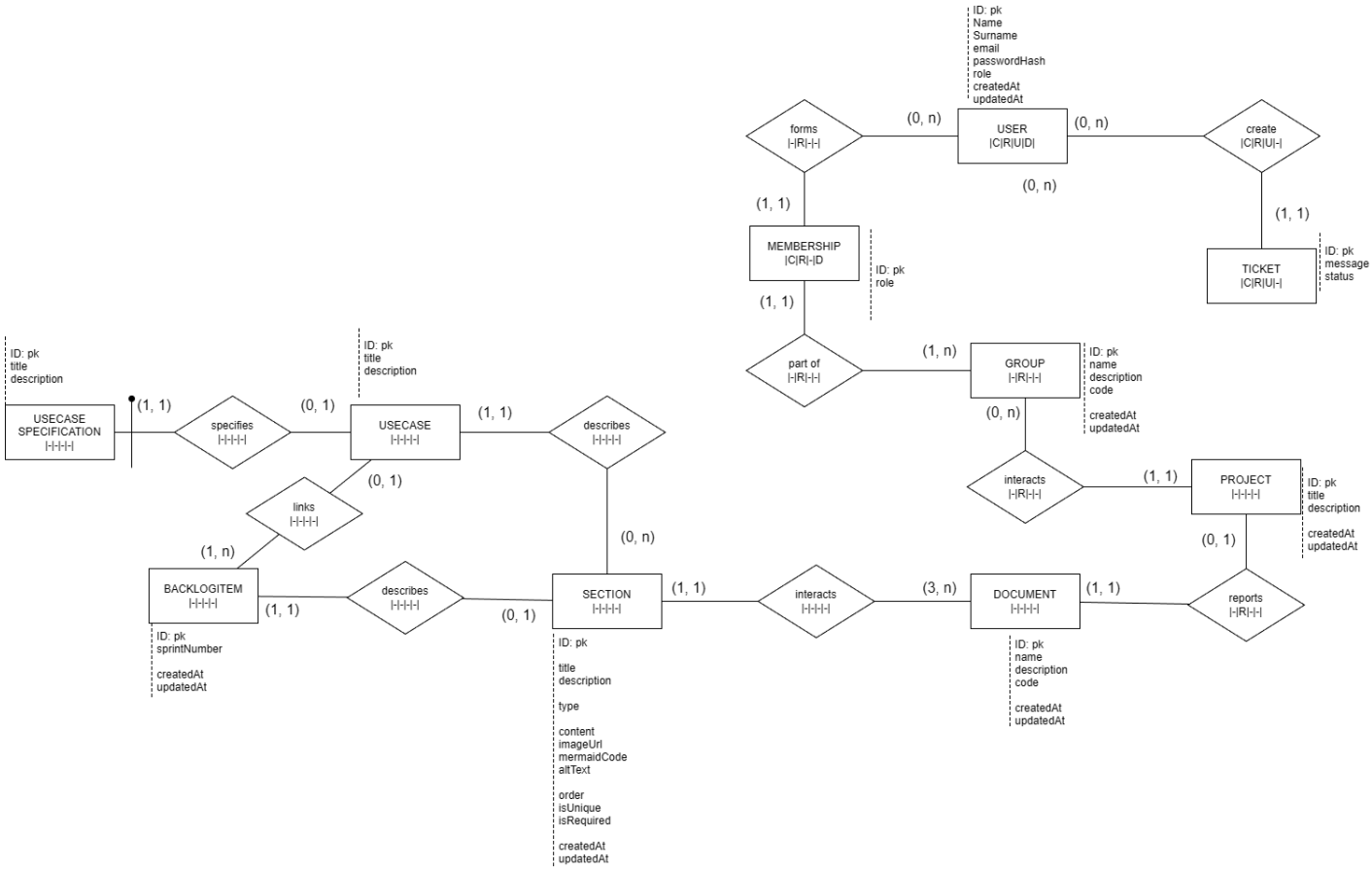
Collezione	Dati	Indici	Totale
User	7.52 MB	1.70 MB	9.22 MB
Ticket	0.80 MB	120 MB	120.80 MB
create	0.80 MB	120 MB	120.80 MB
forms	3.50 MB	2 MB	5.50 MB
membership	0.80 MB	2 MB	2.80 MB
part of	1 MB	2.70 MB	3.70 MB
group	12 MB	3 MB	15 MB
interacts	800 MB	80 MB	880 MB
project	400 MB	120 MB	520 MB
reports	15 MB	50 MB	65 MB
document	3 GB	160 MB	3.16 MB
interacts	4.2 GB	120 MB	4.32 GB
section	2.5 GB	150 MB	2.65 GB
describes	800 mB	70 MB	870 MB
usecase	2 GB	200 MB	2.20 GB
lists	20 MB	0.8 MB	20.80 MB
backlogitem	50 MB	20 MB	70 MB
links	15 MB	1 MB	16 MB
specifies	40 MB	8 MB	48 MB
usecase specification	3 GB	200 MB	3.20 GB
Totale			~15.14 GB

4.3.3 Crescita annuale

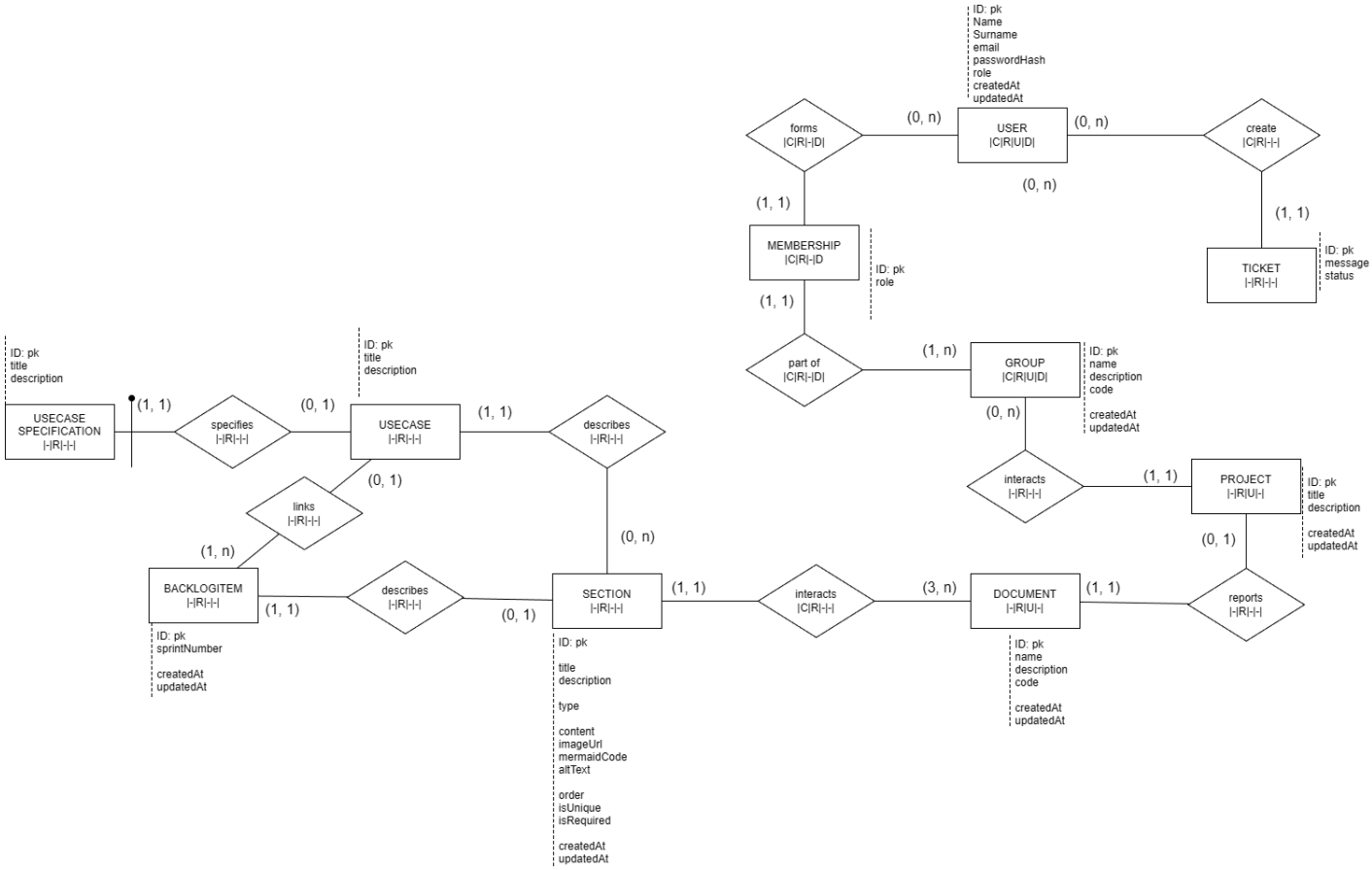
- Anno 1: 15.14 GB
- Anno 2: 6.05 GB (40%)
- Anno 3: 5.30 GB (35% crescita organica degli utenti)

4.4 Modello di visibilità

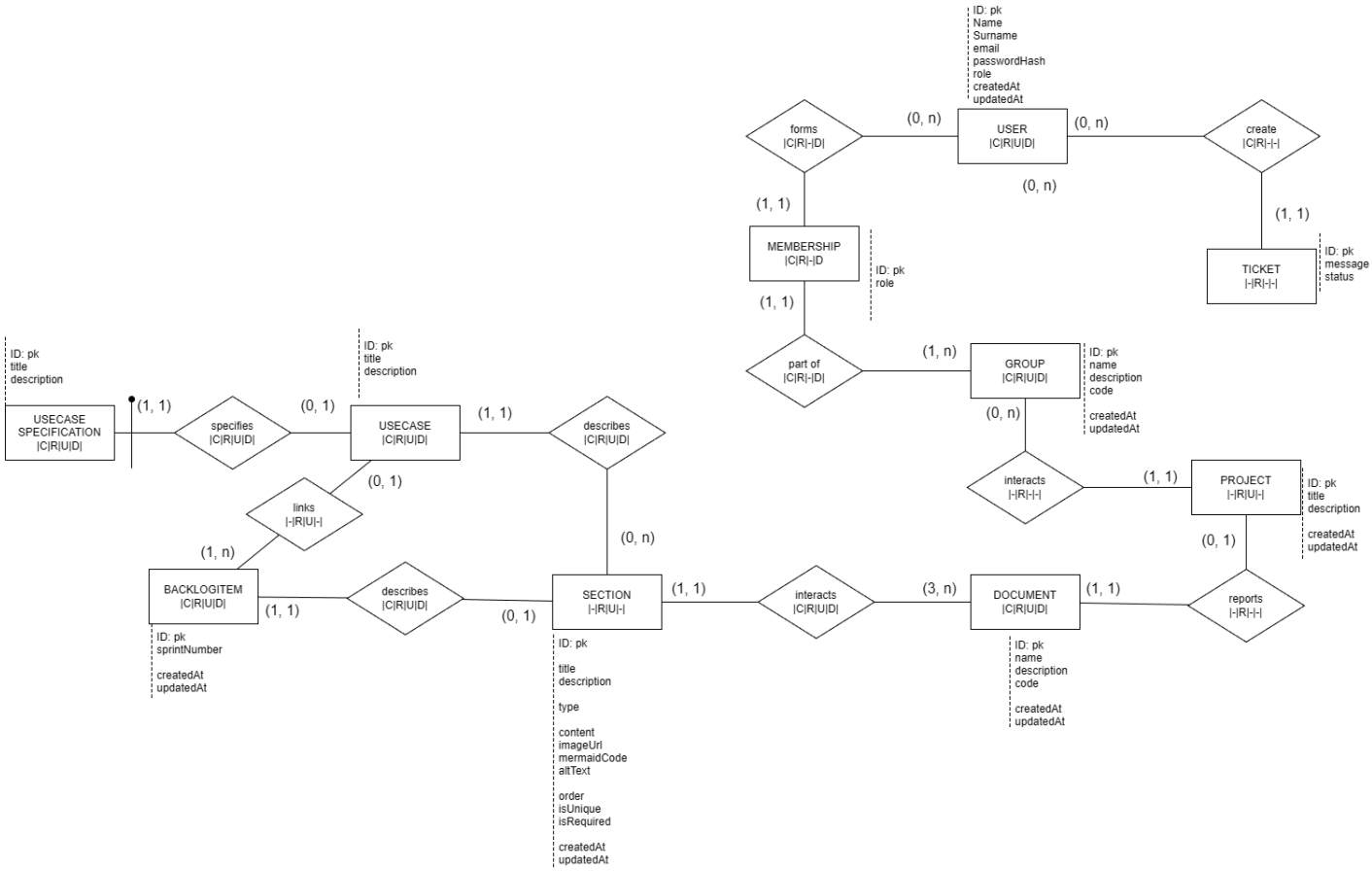
4.4.1 Visibilità dell'amministratore



4.4.2 Visibilità del product manager

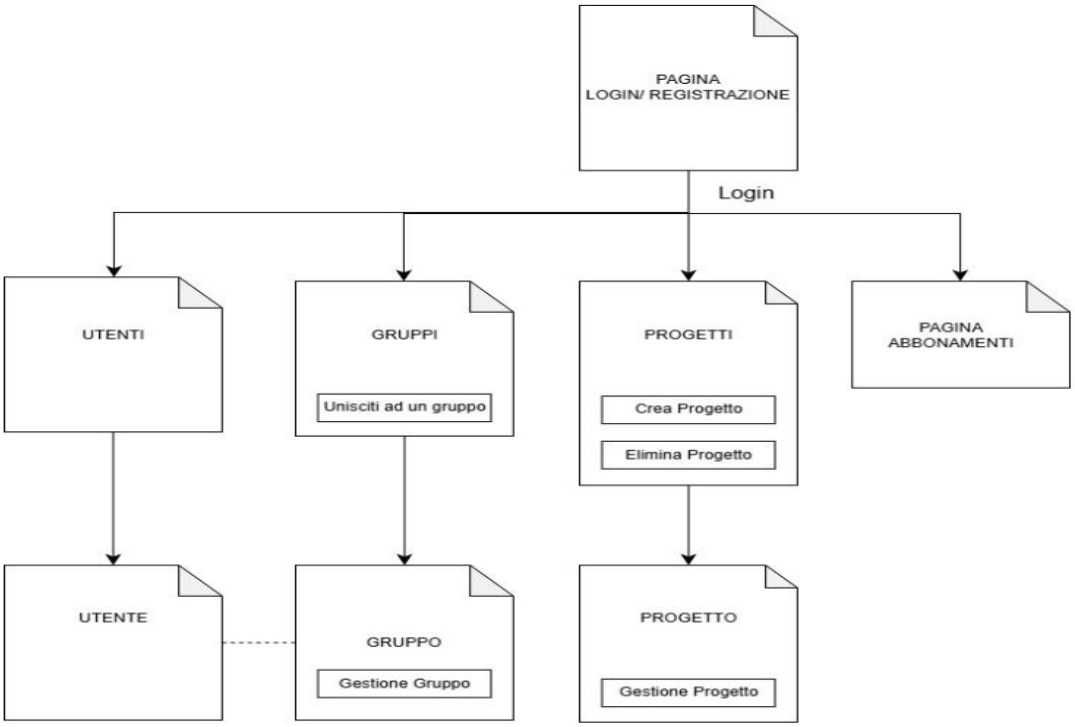


4.4.3 Visibilità del software engineer



5 MODELLO DI NAVIGAZIONE E PRESENTAZIONE

5.1 Modello di navigazione



5.2 Modello di presentazione

L'interfaccia della web application è progettata per supportare un flusso di lavoro strutturato e tracciabile nella gestione della documentazione di progetto. L'obiettivo della presentazione è ridurre la frammentazione informativa e guidare l'utente nella consultazione e compilazione degli item (requisiti, casi d'uso, modello dati), mantenendo coerenza e immediatezza d'uso.

Layout e struttura globale

L'applicazione adotta un layout "dashboard" con sidebar di navigazione a sinistra e area contenuti a destra. La sidebar è progettata per rimanere sempre disponibile anche durante lo scroll), in modo da mantenere continuità di navigazione e ridurre i cambi di contesto. La navigazione è organizzata in gruppi logici: una sezione "core" (es. Home, Users, Private Area) separata da una sezione "business/public" (es. Product, Solution, Templates, Pricing), così da rendere immediata la distinzione tra funzionalità operative e contenuti informativi.

Componenti UI riutilizzabili

Per garantire coerenza visiva e velocità di sviluppo, la UI si basa su componenti standardizzati e riutilizzabili:

- Sidebar/Nav con evidenza della sezione attiva e grouping delle voci;
- Header/topbar per azioni rapide, profilo utente e contesto;
- Cards per riepiloghi e KPI informativi;
- Tabelle con ricerca e filtri per elenchi (es. utenti, progetti, item);
- Form guidati per inserimento/modifica, con validazione e messaggi di errore;
- Metodi di conferma per azioni distruttive (es. eliminazione);
- Notifiche (toast/alert) per feedback immediato su operazioni completate o fallite;

- Stati di caricamento e messaggi di errore per migliorare la qualità percepita.

Template di schermata

Le schermate sono progettate secondo template ricorrenti, per minimizzare la curva di apprendimento:

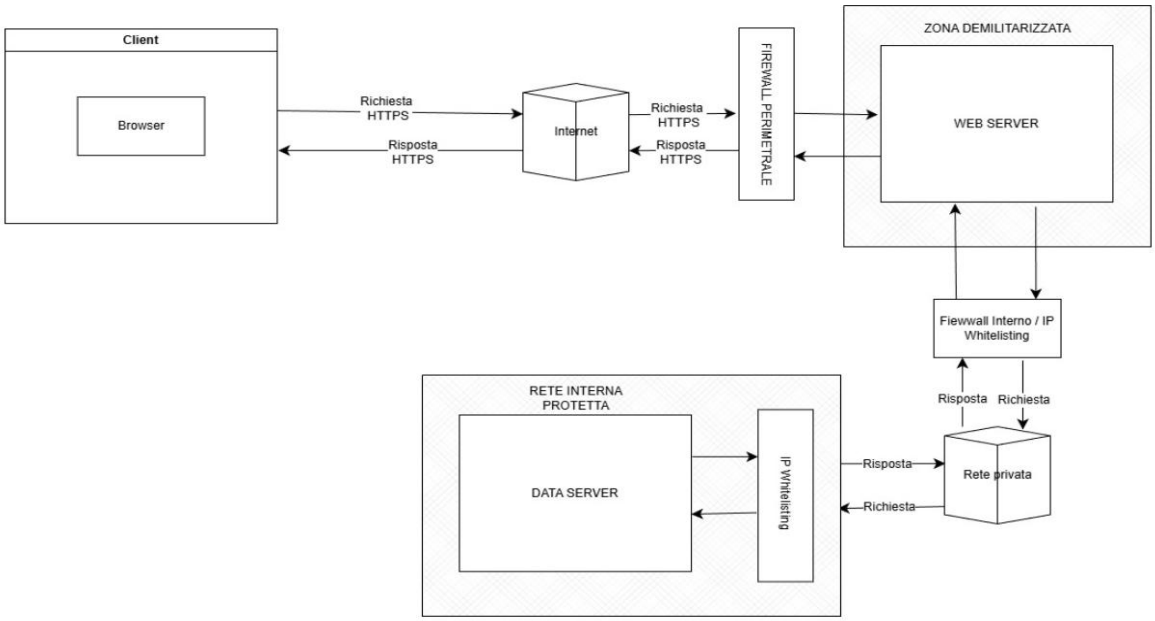
1. Pagine pubbliche (Home/Product/Solution/Templates/Pricing): struttura informativa, contenuti sintetici e orientati alla comprensione del valore.
2. Area autenticata (Private Area/Dashboard): accesso rapido ai progetti e agli item, visualizzazione di stato e attività recenti.
3. Schermate CRUD: liste + dettaglio/modifica, con pattern coerente (tabella, ricerca, pulsante “Create”, form, validazioni).
4. Area amministrativa (Users): gestione utenti e ruoli con viste tabellari, filtri e controlli di sicurezza.

Coerenza con modello di visibilità e ruoli

La presentazione recepisce i ruoli applicativi: le voci di menu e le azioni disponibili variano in base al profilo utente (es. amministratore vs utente standard), riducendo errori operativi e mostrando solo funzionalità rilevanti per il ruolo. Questo rafforza il controllo e la qualità dei dati inseriti, coerentemente con l’obiettivo di mantenere una fonte unica e affidabile per gli artefatti di progetto.

6 MODELLO ARCHITETTURALE

6.1 Hardware



6.1.1 Architettura Hardware

L'architettura hardware è stata progettata secondo il modello **client-server**, con una standard configurazione **Three-tier** ed integrando la sicurezza attraverso una **Demilitarized Zone (DMZ)**

6.2 Software

Il software è realizzato secondo il modello **stack PRN** (PostgreSQL, React, Node.js), che garantisce semplicità d'uso e manutenibilità

6.2.1 Application server

Riceve richieste dal client, restituendo ed elaborando i dati del database, filtrando i dati ed garantendo una forte sicurezza di accesso ai dati

1. Node.js (v.22.14.0)

6.2.2 Librerie principali

Front-end

- **@Next.js** (v. 25.1.09) -> Framework React per applicazioni web full-stack. Gestisce routing, rendering, ottimizzazione delle performance e build/deploy.
- **@React** (v.19.x) -> Libreria UI a componenti. Gestisce rendering dichiarativo, stato, props, lifecycle, hooks e composizione dei componenti.
- **@React DOM** (v.19.x) -> Runtime per il rendering di React nel browser (integrazione con il DOM, hydation lato client)

- **ESLint** (coonfig Next)-> static analysis per individuare errori, anti-pattern e incoerenze di stile, migliora la qualità del codice riducendo bug banali
- **CSS modules** -> Scoping locale degli stili per componente, evitando conflitti globali e rendendo i fogli css più gestibili e modulari
- **Global CSS** -> Stiili globali applicati a tutta l'app
- **Fetch API** (nativa)-> Esecuzione di richieste http verso il backend. In modalità cookie-session viene usata per inviare automaticamente il cookie di sessione

Back-end

- **@prisma/client** (v. 6.19.2) -> Client runtime generato da Prisma per interagire con il database tramite query tipizzate. Consente operazioni CRUD, gestione delle relazioni e accesso ai dati in modo sicuro e integrato con TypeScript
- **Bcryptjs** (v. 3.0.3) -> Libreria JavaScript per l'hashing sicuro delle password. Implementa l'algoritmo bcrypt per proteggere le credenziali degli utenti prima della memorizzazione nel database
- **Crypto** (v. 1.0.1) -> Modulo per operazioni crittografiche (hash, cifratura, generazione di chiavi e token). Utilizzato per garantire integrità dei dati, sicurezza e protezione delle informazioni sensibili
- **Jsonwebtoken** (v. 9.0.3) -> Libreria per creare, firmare e verificare JSON Web Token (JWT). Viene usata per autenticazione e autorizzazione stateless nelle applicazioni web e nelle API
- **Next** (v. 16.1.6) -> Framework React per lo sviluppo di applicazioni web full-stack. Supporta rendering lato server (SSR), static generation, routing, API routes e ottimizzazione delle performance.
- **Prisma** (v. 6.19.2) -> ORM e toolkit per la gestione del database. Permette di definire lo schema dei dati, generare migrazioni, sincronizzare il database e produrre automaticamente il client di accesso ai dati.
- **Resend** (v. 6.9.1) -> Servizio e SDK per l'invio di email transazionali (es. conferma registrazione, reset password) integrato con applicazioni web e backend.
- **Openai** (v. 6.18.0) -> SDK ufficiale per interagire con i modelli di intelligenza artificiale di OpenAI (es. GPT). Consente generazione di testo, automazioni, chatbot e integrazioni AI lato server.

- **Database Management System**

- **PostgreSQL** (v. 17.4) -> Sistema di gestione di database relazionale open source, basato su SQL. Offre transazioni ACID, supporto a relazioni complesse, estensioni avanzate e alte prestazioni per applicazioni enterprise e web

- **Browser supportati**

- **Sistemi operativi supportati**

- **Android** (v. 8.0+)
- **iOS** (v. 13+)
- **Windows** (10, 11)
- **macOS** (Big Sur 11+)
- **Linus** (distribuzioni moderne)

6.2.3

7 TEST

7.1 Strategia di Test

Il sistema è stato testato attraverso un approccio manuale.

7.2 Test Funzionali

7.2.1 Autenticazione e autorizzazione

Caso di test	Tipo	Esito
Registrazione nuovo utente (admin)	Manuale	positivo
Registrazione nuovo utente (Base)	Manuale	positivo
Registrazione nuovo utente (Upgrade)	Manuale	positivo
Login con credenziali valide	Manuale	positivo
Login con credenziali invalide	Manuale	positivo
Validazione persistenza cookie	Manuale	positivo
Validazione hashing della password	Manuale	positivo
Accesso a funzioni non autorizzate (product manager)	Manuale	positivo
Accesso a funzioni non autorizzate (software engineer)	Manuale	positivo

7.2.2 Gestione Progetti
7.2.3 Gestione Gruppi
7.2.4 Gestione Documento
7.2.5 Gestione Sezione
7.2.6 Gestione Statistiche

7.3 Test non Funzionali

7.3.1 Performance
7.3.2 Usabilità
7.3.3 Robustezza

7.4 Test di Compatibilità

7.4.1 **Browser** eseguito e verificato in chrome/edge su windows
7.4.2 **Routing App Router** compatibilità con next.js 25

7.5 Strumenti di Test Utilizzati

7.5.1 **Postman**: Test API REST (CRUD, autenticazione)
7.5.2 **pgAdmin**: Verifica dell'integrità dei dati e manipolazioni via query
7.5.3 **DevTools**: Lettura errori runtime, controllo di routing/404, controlli fetch
7.5.4 **Next.js dev server:** Ambiete di test per routing e rendering

Tutti i test effettuati sul FrontEnd sono stati fatti attraverso la modalità DEV, ovvero creando utenti fittizi al fine di testare l'interfaccia in tempo reale
Tutti i test effettuati dopo il collegamento tra FrontEnd e BackEnd sono stati svolti attraverso la creazione di veri utenti e quindi, di creazione account con dati veritieri. Tutti gli utenti registrati sono stati informati, previa comunicazione scritta, che i loro account potranno essere oggetto di test sperimentali e di presentazioni per gli acquirenti. Inoltre, tutti gli account registrati in fase demo sono GRATUITI

8 CONCLUSIONI

Con questo progetto abbiamo sviluppato una web application che funge da punto unico di raccolta e gestione della documentazione di un progetto software. L'obiettivo è evitare che requisiti, casi d'uso, modello dati e specifiche finiscano sparsi in file separati, difficili da aggiornare e ancora più difficili da mantenere coerenti. L'idea alla base è semplice ma concreta: mettere tutto in un unico posto, collegare le informazioni tra loro in modo chiaro e rendere la consultazione più immediata per chiunque debba lavorarci, come sviluppatori e project manager, con i ruoli specifici associati. In questo modo si riducono errori, duplicazioni e dimenticanze che spesso emergono nelle fasi di analisi e progettazione, inoltre si riducono drasticamente condivisioni di file inerenti, portando un minor rischio di perdita di documenti o perdita di aggiornamenti funzionali.

Dalla documentazione prodotta emerge una soluzione strutturata e completa: contesto e obiettivi, ruoli e responsabilità, analisi delle alternative, modello informativo, navigazione, architettura, aspetti implementativi e test. Non si tratta di una semplice demo, ma di un sistema pensato con logica, ordine e tracciabilità, in linea con ciò che ci si aspetta da un software specifico.

Guardando avanti, i passi più naturali riguardano il miglioramento dell'esperienza d'uso e l'introduzione di maggiore automazione, come esportazioni più ricche, controlli di qualità più rigorosi e magari anche un supporto intelligente integrato. L'obiettivo rimane lo stesso: trasformare la documentazione in uno strumento vivo, aggiornabile e realmente utile, e non in un file statico destinato a diventare obsoleto in poco tempo.

Il team di sviluppo ha lavorato in modo collaborativo, suddividendo le attività in base alle competenze e mantenendo un confronto costante durante tutte le fasi del progetto. Ogni membro ha contribuito con un ruolo specifico, dall'analisi dei requisiti alla progettazione del modello dati, fino all'implementazione e ai test, garantendo un flusso di lavoro ordinato e coerente. Questo approccio ha permesso di affrontare le criticità in modo rapido, condividere soluzioni e mantenere una visione comune sugli obiettivi. La collaborazione continua è stata uno degli elementi chiave che ha reso possibile la realizzazione di un prototipo solido, ben documentato e allineato alle aspettative del progetto.

L'obiettivo finale è rendere questo progetto uno strumento indispensabile per ingegneri e progettisti, aiutandoli a ottimizzare il tempo di lavoro e a mantenere un profilo professionale e organizzato in ogni attività, senza lasciare nulla al caso.

9 MATRICE RACI

La seguente tabelle descrive la distribuzione delle responsabilità per ogni attività dle progetto secondo il modello RACI

- **R (responsabile):** chi segue fisicamente l'attività
- **A (accountable):** chi ha la responsabilità ultima e approva il lavoro
- **C (consulted):** chi viene consultato durante l'esecuzione
- **I (informed):** chi viene informato a lavoro concluso

Attività / Deliverable	**797163 Samuele Belardinelli**	**801933 Ennio Lo Buono**
Descrizione contesto e obbiettivi	R	A/C/I
Individuazione degli stakeholder	R/A	C/I
Diagramma di contesto	R/A	C/I
Scambio di valore	C/I	R/A
Stato dell'arte	A/R	C/I
Analisi "Make Or Buy"	A/R	C/I
Analisi della concorrenza	A/R	C/I
Diagramma ER	C/I	A/R
Stima dimensionale del DB	C/I	A/R
Document Mode MongoDB	I	A/R/C
Modello di visibilità	A/R	A/R
Modello di navigazione	A/R	A/R
Modello archietturale	A/R	A/R
Aspetti implementativi FrontEnd	A/R	C/I
Aspetti implementativi BackEnd	C/I	A/R
Test	A/R	A/R
Conclusioni	A/R	C/I